

SQL SERVER

String Operations in SQL Server

Complete Guide — All Functions & Techniques



Lesson Agenda — 1 Hour Road Map

01

String Data Types

CHAR, VARCHAR, NVARCHAR, TEXT

02

Length & Case Functions

LEN, DATALENGTH, UPPER, LOWER

03

Trimming & Padding

LTRIM, RTRIM, TRIM, SPACE, REPLICATE

04

Search & Position

CHARINDEX, PATINDEX, LIKE

05

Extraction

SUBSTRING, LEFT, RIGHT

06

Modification

REPLACE, STUFF, REVERSE, CONCAT

07

Conversion & Format

STR, FORMAT, CAST, CONVERT

08

String Splitting & Agg

STRING_SPLIT, STRING_AGG

09

Advanced Patterns

Wildcards, Collations, Best Practices

String Data Types

Understanding how SQL Server stores text — the foundation of everything

01 String Data Types in SQL Server

Data Type	Storage	Unicode	Max Size	Best Practice
CHAR(n)	n bytes (fixed)	No	8,000	Pads with spaces. Use for fixed-length codes (e.g. country codes).
VARCHAR(n)	actual length + 2	No	8,000	Variable length. Best for general text.
VARCHAR(MAX)	up to 2 GB	No	2 GB	Large text. Avoid in indexed columns.
NCHAR(n)	2n bytes (fixed)	Yes	4,000	Unicode fixed-length. Use for multilingual fixed data.
NVARCHAR(n)	2× actual + 2	Yes	4,000	Unicode variable. Recommended for multilingual apps.
NVARCHAR(MAX)	up to 2 GB	Yes	2 GB	Large Unicode text.

-- Examples

```
DECLARE @code CHAR(2)      = 'IN';  
DECLARE @name VARCHAR(100) = 'Rahul Sharma';  
DECLARE @bio  NVARCHAR(500) = N'नमस्ते - Hello';
```

Length & Case Functions

LEN, DATALENGTH, UPPER, LOWER

02 Length & Case — Functions & Examples

LEN(string)

Returns number of characters, excluding trailing spaces.

LEN('Hello ') → 5

DATALENGTH(string)

Returns bytes used in storage — Unicode = 2× chars.

DATALENGTH(N'Hi') → 4 bytes

UPPER/LOWER(string)

Converts characters to upper or lower case.

UPPER('sql') → 'SQL' | LOWER('SQL') → 'sql'

```
DECLARE @s VARCHAR(20) = 'SQL Server';
DECLARE @n NVARCHAR(20) = N'SQL Server';
SELECT LEN(@s) AS LEN_varchar,      -- 10
       DATALENGTH(@s) AS Bytes_varchar,  -- 10
       LEN(@n) AS LEN_nvarchar,      -- 10
       DATALENGTH(@n) AS Bytes_nvarchar; -- 20 (Unicode = 2 bytes/char)
```

► Output

LEN_varchar = 10 | Bytes_varchar = 10 | LEN_nvarchar = 10 | Bytes_nvarchar = 20

-- Case Functions

```
SELECT
    UPPER('hello world') AS upper_result,  -- HELLO WORLD
    LOWER('HELLO WORLD') AS lower_result,  -- hello world
    LOWER(UPPER('Mixed')) AS combined;     -- mixed
```

► Output

upper_result = 'HELLO WORLD' | lower_result = 'hello world' | combined = 'mixed'

Trimming & Padding

LTRIM, RTRIM, TRIM, SPACE, REPLICATE

03 Trimming & Padding — Clean Your Data



LTRIM(s)

Remove leading spaces
LTRIM(' Hi') → 'Hi'



RTRIM(s)

Remove trailing spaces
RTRIM('Hi ') → 'Hi'



TRIM([chars FROM] s)

Remove both sides (SQL 2017+)
TRIM(' Hi ') → 'Hi'



SPACE(n)

Returns n space characters
SPACE(5) → ' '



REPLICATE(s,n)

Repeats string n times
REPLICATE('AB',3) → 'ABABAB'

-- Real-world: Clean messy customer data

```
SELECT
    LTRIM(RTRIM(customer_name))      AS clean_name,
    TRIM('.') FROM email              AS clean_email,
    REPLICATE('*', 8) + RIGHT(phone, 4) AS masked_phone,
    SPACE(3) + first_name             AS indented_name
FROM Customers;
```

-- Pad a number to 6 digits: '42' → '000042'

Search & Position

CHARINDEX, PATINDEX, LIKE wildcards

04 Search & Position — Finding Text

CHARINDEX(find, string [, start])

Returns position of first match (1-based). Returns 0 if not found.
CHARINDEX('ql', 'SQL Server') → 2

PATINDEX('%pattern%', string)

Like CHARINDEX but supports LIKE wildcards. Returns 0 if no match.
PATINDEX('%[0-9]%', 'Room 5B') → 6

-- CHARINDEX examples

```
SELECT
    CHARINDEX('Server', 'SQL Server 2022') AS pos1, -- 5
    CHARINDEX('x', 'SQL Server') AS pos2, -- 0 (not found)
    CHARINDEX('s', 'SQL Server', 4) AS pos3; -- 5 (start from pos 4)
```

-- PATINDEX with wildcards

```
SELECT
    PATINDEX('%[0-9]%', 'Invoice INV-00123') AS first_digit, -- 13
    PATINDEX('%@%.%', 'user@example.com') AS email_check; -- 5
```

► Output

pos1 = 5 | pos2 = 0 | pos3 = 5 | first_digit = 13 | email_check = 5

LIKE Wildcards Quick Reference

%	Any sequence of zero or more chars	LIKE 'mysql%' → matches 'MySQL', 'SQL Server'
_	Any single character	LIKE 'c_t' → matches 'cat', 'cut', 'cot'
[abc]	Any one of listed chars	LIKE '[abc]%' → starts with a, b, or c
[^abc]	Any char NOT in list	LIKE '[^0-9]%' → starts with non-digit

String Extraction

SUBSTRING, LEFT, RIGHT

05 Extraction — Getting Parts of a String

SUBSTRING(string, start, length)

Extracts length chars starting at start (1-based).

SUBSTRING('SQL Server', 5, 6) → 'Server'

LEFT(string, n)

First n characters from the left.

LEFT('SQL Server', 3) → 'SQL'

RIGHT(string, n)

Last n characters from the right.

RIGHT('SQL Server', 6) → 'Server'

```
-- Basic extraction
```

```
SELECT
```

```
    SUBSTRING('SQL Server 2022', 1, 3) AS part1,    -- SQL
    SUBSTRING('SQL Server 2022', 5, 6) AS part2,    -- Server
    LEFT('john.doe@email.com', 8)      AS leftPart, -- john.doe
    RIGHT('INV-00123', 5)              AS invoice;  -- 00123
```

```
-- Extract domain from email (dynamic length)
```

```
DECLARE @email VARCHAR(100) = 'student@example.com';
```

```
SELECT
```

```
    LEFT(@email, CHARINDEX('@', @email) - 1)          AS username, -- student
    SUBSTRING(@email, CHARINDEX('@', @email) + 1, LEN(@email)) AS domain; -- example.com
```

Output

```
part1='SQL' | part2='Server' | leftPart='john.doe' | invoice='00123'
username='student' | domain='example.com'
```

 Pro Tip: Combine CHARINDEX + SUBSTRING to extract dynamic segments between delimiters (e.g. slicing 'FirstName LastName' on the space character).

String Modification

REPLACE, STUFF, REVERSE, CONCAT, CONCAT_WS

06 Modification — Changing String Content

REPLACE(s, old, new)

Replace all occurrences of old with new.

```
REPLACE('Hello World','World','SQL') → 'Hello SQL'
```

STUFF(s, start, len, new)

Delete len chars at start and insert new string.

```
STUFF('SQL2019',4,4,'2022') → 'SQL2022'
```

REVERSE(s)

Reverses the character order.

```
REVERSE('SQL') → 'LQS'
```

CONCAT(s1,s2,...)

Join strings — NULLs treated as empty string.

```
CONCAT('Hello',' ','World') → 'Hello World'
```

CONCAT_WS(sep,s1,s2,...)

Join with separator, skipping NULLs (2017+).

```
CONCAT_WS(', ','Delhi',NULL,'India') → 'Delhi, India'
```

06 Modification — Practical Examples

```
-- REPLACE: sanitize phone numbers
SELECT REPLACE(REPLACE(REPLACE(phone_no, '-', ''), ' ', ''), '(', '' ) AS clean_phone
FROM Customers;    -- '(91) 98-765-4321' → '919876543 21'

-- STUFF: mask credit card
DECLARE @card VARCHAR(20) = '411111111111234';
SELECT STUFF(@card, 1, 12, REPLICATE('*',12)) AS masked;    -- *****1234

-- REVERSE: check palindrome
DECLARE @word VARCHAR(50) = 'racecar';
SELECT CASE WHEN @word = REVERSE(@word) THEN 'Palindrome' ELSE 'Not' END AS result;

-- CONCAT_WS: build full address
SELECT CONCAT_WS(' ', house_no, street, city, state, pincode) AS full_address
FROM Addresses;    -- '12, MG Road, Surat, Gujarat, 395001'
```

► Output

masked = '*****1234'

result = 'Palindrome'

full_address = '12, MG Road, Surat, Gujarat, 395001'

⚠ Common Mistake — String Concatenation with NULL

```
-- NULL kills the whole result with + operator:
SELECT 'Hello' + NULL + 'World'    -- NULL ❌

-- Use CONCAT which ignores NULL:
SELECT CONCAT('Hello', NULL, 'World')    -- 'HelloWorld' ✅
```

Conversion & Formatting

STR, FORMAT, CAST, CONVERT

07 Conversion & Formatting

```
-- STR: convert numeric to string (fixed-width, right-aligned)
SELECT STR(3.14159, 8, 2)    AS result; -- '    3.14' (width=8, decimals=2)

-- FORMAT: rich formatting using .NET format strings (SQL 2012+)
SELECT
    FORMAT(GETDATE(), 'dd-MMM-yyyy')           AS date_str,    -- 15-Mar-2025
    FORMAT(1234567.89, 'N2', 'en-IN')          AS amount,     -- 12,34,567.89
    FORMAT(0.1575, 'P1')                       AS pct;         -- 15.8%

-- CAST vs CONVERT
SELECT
    CAST(42 AS VARCHAR(10))                   AS cast_ex,     -- '42'
    CONVERT(VARCHAR, GETDATE(), 103)           AS date_uk,     -- 15/03/2025
    CONVERT(VARCHAR, GETDATE(), 120)           AS iso_fmt,      -- 2025-03-15 10:30:00
    TRY_CAST('abc' AS INT)                    AS safe_cast;    -- NULL (no error)
```

```
► Output= '    3.14' | date_str='15-Mar-2025' | amount='12,34,567.89' | pct='15.8%'
date_uk='15/03/2025' | iso_fmt='2025-03-15 10:30:00' | safe_cast=NULL
```

Common CONVERT Date Style Codes

101	103	104	112	120
mm/dd/yyyy	dd/mm/yyyy	dd.mm.yyyy	yyyymmdd	yyyy-mm-dd hh:mi:ss
US	UK / India	German	ISO Basic	ODBC / ISO Extended

Splitting & Aggregating

STRING_SPLIT & STRING_AGG

08 STRING_SPLIT & STRING_AGG (SQL Server 2016+/2017+)

STRING_SPLIT(string, separator)

Table-valued function — splits a delimited string into rows.
Returns a table with a value column.

STRING_AGG(expr, separator)

Aggregate — joins rows into one delimited string.
Opposite of STRING_SPLIT. Works like GROUP_CONCAT in MySQL.

```
-- STRING_SPLIT: parse CSV tags
DECLARE @tags VARCHAR(200) = 'SQL,Python,Excel,Power BI';
SELECT value AS tag FROM STRING_SPLIT(@tags, ',');
-- Output rows: SQL | Python | Excel | Power BI

-- Real use: filter rows matching any item in a CSV parameter
SELECT * FROM Products
WHERE category IN (SELECT value FROM STRING_SPLIT('Books,Laptops,Phones', ','));
```

► Output

```
tag
----
SQL
Python
Excel
Power BI
```

```
-- STRING_AGG: build comma-separated list per group
SELECT
    department,
    STRING_AGG(employee_name, ', ')
    WITHIN GROUP (ORDER BY employee_name) AS employees
FROM Employees
GROUP BY department;
```

► Output

department='Engineering'	employees='Arjun, Kavya, Rohan'
department='Sales'	employees='Anjali, Rahul'

Advanced Patterns

Collations, wildcards, performance & best practices

09 Collations — Case & Accent Sensitivity

Collation controls HOW strings are compared and sorted.

_CI_AS

Case Insensitive, Accent Sensitive

```
'SQL' = 'sql' → TRUE  
'cafe' = 'café' → FALSE
```

_CS_AS

Case Sensitive, Accent Sensitive

```
'SQL' = 'sql' → FALSE  
'cafe' = 'café' → FALSE
```

_CI_AI

Case Insensitive, Accent Insensitive

```
'SQL' = 'sql' → TRUE  
'cafe' = 'café' → TRUE
```

_BIN2

Binary — fastest, exact byte compare

Use for codes & passwords
Fastest but strictest

```
-- Override collation in a query  
SELECT name FROM Users  
WHERE name = 'Admin' COLLATE SQL_Latin1_General_CP1_CS_AS; -- case-sensitive check  
  
-- Check column collation  
SELECT COLUMN_NAME, COLLATION_NAME FROM INFORMATION_SCHEMA.COLUMNS  
WHERE TABLE_NAME = 'Customers';
```

09 Best Practices & Performance Tips

Use NVARCHAR for user-facing text

Supports multilingual data. Prefix literals with N: N'नमस्ते' to avoid silent data loss.

Avoid functions on indexed columns in WHERE

WHERE UPPER(email) = 'A@B.COM' prevents index use. Use case-insensitive collation instead.

Right-size your VARCHAR

VARCHAR(MAX) disables some optimizations. Use the smallest size that fits your data.

Always use TRY_CAST / TRY_CONVERT

Prevents runtime errors when converting untrusted string input to numeric/date types.

Clean data at ingestion

Apply LTRIM/RTRIM/TRIM in INSERT or ETL triggers so downstream queries stay fast.

STRING_AGG > FOR XML PATH

STRING_AGG is cleaner and faster than the old FOR XML PATH hack for concatenating rows.

Quick Reference — All String Functions at a Glance

LEN(s)

Character count (no trailing spaces)

DATALength(s)

Byte count

UPPER/LOWER(s)

Change case

LTRIM/RTRIM/TRIM(s)

Remove spaces

SPACE(n)

Generate spaces

REPLICATE(s,n)

Repeat string

CHARINDEX(f,s[,n])

Position of substring

PATINDEX('%p%',s)

Position with wildcards

SUBSTRING(s,n,len)

Extract by position

LEFT/RIGHT(s,n)

From left or right end

REPLACE(s,old,new)

Replace all occurrences

STUFF(s,n,len,new)

Delete then insert

REVERSE(s)

Flip string backwards

CONCAT(s1,s2,...)

Join (NULL→empty)

CONCAT_WS(sep,...)

Join with separator

STRING_SPLIT(s,sep)

Split to rows (2016+)

STRING_AGG(e,sep)

Rows to string (2017+)

CAST/TRY_CAST

Type conversion

CONVERT(type,e,style)

With format style

FORMAT(e,'fmt')

Rich number/date format



Lesson Complete!

You now know all core SQL Server String Operations

✓ String Data Types

✓ Search & LIKE

✓ Conversion

✓ Length & Case

✓ Extraction

✓ Split & Aggregate

✓ Trim & Pad

✓ Modification

✓ Best Practices

Keep Practicing — String mastery = Clean, powerful SQL!